Malware RE1: Easy crackme, keygenme

Đào Ngọc Ánh

crack_me1.exe

Thông tin sơ bộ:

crack_me1.exe

Property	Value
File Name	C:\Users\ngahn\Downloads\crack_me1.exe
File Type	Portable Executable 32
File Info	Microsoft Visual C++ 8
File Size	14.50 KB (14848 bytes)
PE Size	14.50 KB (14848 bytes)
Created	Friday 06 May 2022, 00.37.22
Modified	Friday 06 May 2022, 00.37.22
Accessed	Friday 06 May 2022, 00.54.53
MD5	431F04435D7ACE699288A199B3540A2D
SHA-1	F0EAF3D29A2C8B234A33E39A3C081137CEDC9182

Chương trình yêu cầu nhập vào một input, kiểm tra input, sau đó giải mã một đoạn dữ liệu mã hoá với key là input và in ra đoạn dữ liệu sau khi giải mã:

C:\Windows\System32\cmd.exe

```
Microsoft Windows [Version 10.0.18363.418]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ngahn\Downloads>crack_me1.exe
Do you remember the good old days?! I don't know how about you,but I don't. Please help me to recover my memory, it's
 password protected, and that's sad :(
Enter password: ala
oh, no!
```

```c
int __cdecl main(int argc, const char **argv, const char **envp)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  inp = 0;
  memset(inp_1, 0, sizeof(inp_1));
  v9 = 335;
  memset(v7, 0, sizeof(v7));
  strcpy(
    Format,
    "Do you remember the good old days?! I don't know how about you,but I don't. Please help me to recover my memory, it'"
    "s password protected, and that's sad :(\n");
  v8 = &unk_404BE8;
  printf(Format);
  printf("Enter password: ");
  scanf("%300[^\n]s", (char)&inp);
  p_inp = &inp;
  p_inp = inp_1;
  p_inp += strlen(p_inp);
  len = ++p_inp - inp_1;
  len = p_inp - inp_1;
  p_inp = &inp;
  p_inp = inp_1;
  p_inp += strlen(p_inp);
  len = ++p_inp - inp_1;
  if ( (unsigned int)(p_inp - inp_1) >= 0x126 ) // if (strlen(inp)  >= 0x126)
  {
    if ( (unsigned __int8)checkinp(&inp) )
    {
      v15 = len / 3;
      for ( i = 0; i < len; ++i )
        v7[i % v15] ^= inp_1[i - 1];
      for ( i = 0; i < v9; ++i )
        bytes[i] ^= v7[i % v15];
      printf("\n\nCongratulation! Here is your memo :> \n\n");
      printf("%s");
    }
    else
    {
      printf("\nInvalid password\n");
    }
```

Chi tiết:

- Nhận input tối đa 300 ký tự cho đến khi gặp ký tự \n đầu tiên:

```c
scanf("%300[^\n]s", (char)&inp);
```

- Nếu độ dài input lớn hơn hoặc bằng 0x126 ký tự thì gọi hàm checkinp(&inp):

```c
p_inp = &inp;
p_inp = inp_1;
p_inp += strlen(p_inp);
len = ++p_inp - inp_1;
len = p_inp - inp_1;
p_inp = &inp;
p_inp = inp_1;
p_inp += strlen(p_inp);
len = ++p_inp - inp_1;
if ( (unsigned int)(p_inp - inp_1) >= 0x126 ) // if (strlen(inp)  >= 0x126)
{
  if ( (unsigned __int8)checkinp(&inp) )
```

- Hàm checkinp(a1) kiểm tra độ dài input rồi gọi hàm checkbyte kiểm tra từng ký tự của input:

```
char __cdecl checkinp(const char *a1)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  p_data = &data;
  if ( (int)strlen(a1) < 55 )
    return 0;
  for ( i = 0; i < 122; ++i )
  {
    if ( !(unsigned __int8)checkbyte(*p_data, &a1[p_data[1]], p_data + 2) )
      return 0;
    p_data += 3;
  }
  return 1;
}
```

- checkbyte(option, input, cipher) giải mã một shellcode đã mã hóa xor và thực hiện shellcode. Các shellcode đều có nhiệm vụ mã hóa một phần chuỗi đầu vào input và trả về kết quả so sánh input với phần tương ứng của chuỗi đã mã hóa cipher. Hàm nhận 3 tham số:
    - option: Giá trị để chọn 1 trong 4 shellcode để sử dụng
    - input: Chuỗi đầu vào
    - cipher: chuỗi dữ liệu mã hóa

```c
char __cdecl checkbyte(int option, int p_input, int p_cipher)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v8 = 221;
  shell5 = decodeShellcode((int)&xoredShell5, 221, 5);
  v7 = 278;
  shell6 = decodeShellcode((int)&xoredShell6, 278, 6);
  if ( !shell5 || !shell6 )
    return 0;
  switch ( option )
  {
    case 1:
      dwSize = 97;
      shellcode = decodeShellcode((int)&xoredShell1, 97, 1);
      break;
    case 2:
      dwSize = 142;
      shellcode = decodeShellcode((int)&xoredShell2, 142, 2);
      break;
    case 3:
      dwSize = 1685;
      shellcode = decodeShellcode((int)&xoredShell3, 1685, 3);
      break;
    case 4:
      dwSize = 235;
      shellcode = decodeShellcode((int)&xoredShell4, 235, 4);
      break;
    default:
      return 0;
  }
  if ( !shellcode )
    return 0;
  shell = ((int (__cdecl *)(int, int, char *))shellcode)(p_input, p_cipher, v4);
  VirtualFree(shellcode, dwSize, 0x8000u);
  VirtualFree(shell5, v8, 0x8000u);
  VirtualFree(shell6, v7, 0x8000u);
  return shell;
}
```

```c
_BYTE *__cdecl decodeShellcode(int xoredShellcode, int dwSize, char key)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  if ( dwSize <= 0 )
    return 0;
  keya = key + 32;
  p_shell = VirtualAlloc(0, dwSize, 0x1000u, 0x40u);
  if ( !p_shell )
    return 0;
  for ( i = 0; i < dwSize; ++i )
    p_shell[i] = keya ^ *(_BYTE *)(i + xoredShellcode);
  return p_shell;
}
```

- shellcode1(char input, char byte) nhận một ký tự từ mỗi chuỗi. Shellcode1 mã hoá input dựa trên tính chẵn lẻ và so sánh với cipher:
  - Nếu input là ký tự chẵn: xor input với 20h

- Nếu input là ký tự lẻ: xor với 52h

```
bool __cdecl Shellcode1(char *input, unsigned __int8 *cipher)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v2 = *input % 2;
  if ( !v2 && (*input ^ 0x20) == *cipher )
    return 1;
  return v2 == 1 && (*input ^ 0x52) == *cipher;
}
```

- shellcode 2(input, cipher) nhận 2 ký tự từ mỗi chuỗi và ghép thành 1 WORD. Shellcode2 mã hoá 2 ký tự liên tiếp trong input bằng phép đảo bit và xor, so sánh với a2:

```
bool __cdecl Shellcode2(char *input, unsigned __int16 *cipher)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v4 = input[1] | (unsigned __int16)(*input << 8);
  for ( i = 1; i <= 5; ++i )
    v4 = (((int)v4 >> (16 - i)) | (v4 << i)) ^ 0x1693;
  return *cipher == v4;
}
```

- shellcode 3 nhận 3 ký tự liên tiếp từ input và 4 ký tự liên tiếp từ cipher. Shellcode3 mã hoá 3 ký tự liên tiếp trong input sang base64 với custom charset rồi so sánh kết quả với cipher:

```c
bool __cdecl Shellcode3(char *input, unsigned __int8 *cipher)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v9 = 0;
  i = 0;
  v7 = 3;
  v5 = 0;
  v4 = 0;
  qmemcpy(v3, "ABDCEHGFIJKLUNOPYRTSMVWXQZajcdefohibkmlngpqrstuv4xzy8123w56709+0", 64);
  while ( v7-- )
  {
    *(&v14 + v9++) = *input++;
    if ( v9 == 3 )
    {
      v10 = (v14 & 0xFC) >> 2;
      if ( v3[v10] != *cipher )
        return 0;
      v11 = ((v15 & 0xF0) >> 4) + 16 * (v14 & 3);
      if ( v3[v11] != cipher[1] )
        return 0;
      v12 = ((v16 & 0xC0) >> 6) + 4 * (v15 & 0xF);
      if ( v3[v12] != cipher[2] )
        return 0;
      v13 = v16 & 0x3F;
      if ( v3[v16 & 0x3F] != cipher[3] )
        return 0;
      v9 = 0;
    }
  }
  if ( v9 <= 0 )
    return 1;
  for ( i = v9; i < 3; ++i )
    *(&v14 + i) = 0;
  v10 = (v14 & 0xFC) >> 2;
  if ( v3[v10] != *cipher )
    return 0;
  v11 = ((v15 & 0xF0) >> 4) + 16 * (v14 & 3);
  if ( v3[v11] != cipher[1] )
    return 0;
  v12 = ((v16 & 0xC0) >> 6) + 4 * (v15 & 0xF);
  if ( v3[v12] != cipher[2] )
    return 0;
  v13 = v16 & 0x3F;
  return v3[v16 & 0x3F] == cipher[3];
}
```

- shellcode 4 nhận 4 ký tự liên tiếp từ 2 chuỗi input và cipher, mã hóa input với đoạn dữ liệu được tạo ra từ key “susan” rồi so sánh với cipher.

```c
int __cdecl shellcode4(char *p_input, int cipher, int offsets)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  strcpy(key, "susan");
  createConst = *(void (__cdecl **)(char *, char *))(offsets + 4);
  check = *(int (__cdecl **)(char *, char *, int))(offsets + 8);
  createConst(key, consts);
  input[0] = *p_input;
  input[1] = p_input[1];
  input[2] = p_input[2];
  input[3] = p_input[3];
  input[4] = 0;
  return check(consts, input, cipher);
```

- Hàm createConst() tạo ra một đoạn dữ liệu từ key “susan” và lưu kết quả vào chuỗi output:

```
int __cdecl createConst(const char *key, int output)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v2 = strlen(key);
  v4 = 0;
  for ( i = 0; i < 256; ++i )
    *(_BYTE *)(i + output) = i;
  for ( j = 0; j < 256; ++j )
  {
    v4 = (key[j % v2] + v4 + *(unsigned __int8 *)(j + output)) % 256;
    v7 = *(_BYTE *)(j + output);
    *(_BYTE *)(j + output) = *(_BYTE *)(v4 + output);
    *(_BYTE *)(v4 + output) = v7;
  }
  return 0;
}
```

- Hàm check() mã hóa xor input với key là hàm output trên và so sánh với cipher:

```
char __cdecl check(int consts, const char *input, int cipher)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  i = 0;
  val = 0;
  j = 0;
  len = strlen(input);
  while ( j < len )
  {
    i = (i + 1) % 256;
    val = (val + *(unsigned __int8 *)(i + consts)) % 256;
    v8 = *(_BYTE *)(i + consts);
    *(_BYTE *)(i + consts) = *(_BYTE *)(val + consts);
    *(_BYTE *)(val + consts) = v8;
    if ( *(unsigned __int8 *)(j + cipher) != (*(unsigned __int8 *)(consts
                                                                   + (*(unsigned __int8 *)(val + consts)
                                                                    + *(unsigned __int8 *)(i + consts))
                                                                   % 256) ^ input[j]) )
      return 0;
    ++j;
  }
  return 1;
}
```

  - Shell5, shell6 được sử dụng để kiểm tra hoạt động của hàm decodeShellcode() trước khi giải mã các shellcode trên.

- Nếu input đúng, chương trình sử dụng input để giải mã đoạn dữ liệu đã bị mã hoá bằng phép xor rồi in ra kết quả:

```
if ( checkinp(&inp) )
{
  v15 = len / 3;
  for ( i = 0; i < len; ++i )
    v7[i % v15] ^= inp_1[i - 1];
  for ( i = 0; i < v9; ++i )
    bytes[i] ^= v7[i % v15];
  printf("\n\nCongratulation! Here is your memo :> \n\n");
  printf("%s");
}
else
{
  printf("\nInvalid password\n");
}
```

Sau khi giải mã, input cần tìm là:

ThiS 1s A rIdiCuLously l0ng_Lon9_l0ng_loNg_lOng strIng. The most difficult thing is the decision to act, the rest is merely tenacity. The fears are paper tigers. You can do anything you decide to do. You can act to change and control your life; and the procedure, the process is its own reward.

Chạy lại chương trình với input này và ta nhận được kết quả:

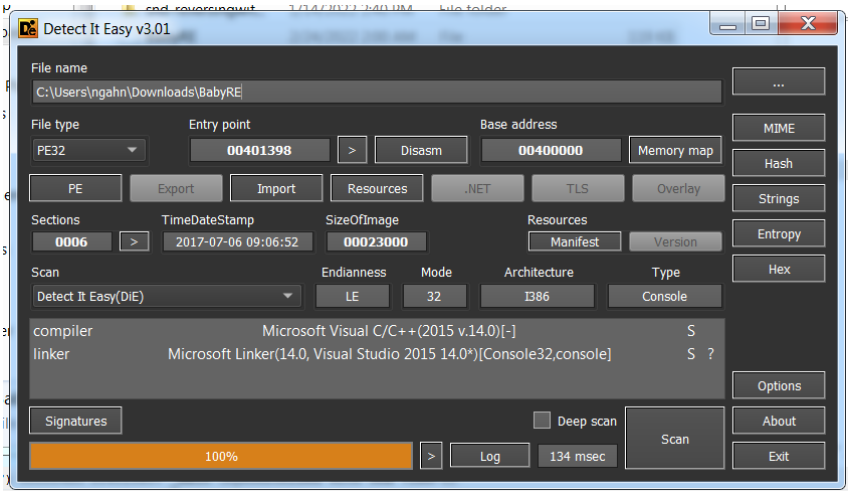
```
C:\Users\ngahn\Downloads\crack_me1.exe
Do you remember the good old days?! I don't know how about you,but I don't. Please help me to recover my memory, it's pa
ssword protected, and that's sad :(
Enter password: ThiS 1s A rIdiCuLously l0ng_Lon9_l0ng_loNg_lOng strIng. The most difficult thing is the decision to act,
 the rest is merely tenacity. The fears are paper tigers. You can do anything you decide to do. You can act to change an
d control your life; and the procedure, the process is its own reward.


Congratulation! Here is your memo :>

#######!!!
```

BabyRE

Thông tin sơ bộ:



- File PE32, compiler MSVC

Chương trình nhận input gồm 5 ký tự từ bàn phím, giải mã flag bằng phép xor rồi in ra flag:

```
int __cdecl memset(int argc, const char **argv, const char **envp)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  flag = *(_OWORD *)"bdnpQai|nufimnug";
  v9 = 1182494304;
  v10 = 1919444577;
  v11 = 1111;
  printf((int)"Guest your flag. The flag will be of the form flag{[a-zA-Z0-9]+}\n");
  printf((int)"Enter your key:");
  for ( i = 0; i < 5; ++i )
    scanf("%hhd", &input[i]);
  for ( j = 0; j < 26; p_flag[1] ^= input[j - 5 * ((unsigned int)&p_flag[1 - (_DWORD)&flag] / 5) + 1] )
  {
    p_flag = (char *)&flag + j;
    *p_flag ^= input[j % 5u];
    j += 2;
  }
  printf("flag is : %s\n", &flag);
  return 0;
}
```

Flag được mã hóa với công thức:

flag[i] = flag[i % 5] ^ input[i]

Từ flag form được cho, ta biết 5 ký tự đầu là “flag{“ và từ đó tìm ra key:

key = flag ^ “flag{“

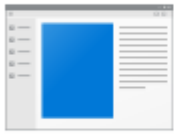
Tìm key và giải mã flag: [script solve]

Flag: flag{easy_baby_challenge}

Hidden Treasure

Đề bài bao gồm một chương trình HiddenTreasure.exe là trình giấu tin vào một file BMP, một file BMP đã được giấu tin.

his PC › Downloads › HiddenTreasure



HiddenTreasure.exe

inside-the-mind-of-a-hacker-memory.bmp

Phân tích file inside-the-mind-of-a-hacker-memory.bmp của đề bài:

```
9d141ee9a7cc8a1f4a5e-eb93ed3a14cfdd3a53b723f6bf84ad5f1f2e7d3f ./a.out inside-the-mind-of-a-hacker-memory.bmp
File type          = BM
File size          = 1633334 bytes
Data offset        = 54 bytes
Info header size   = 40 bytes
Width              = 938 pixels
Height             = 580 pixels
Planes             = 1
Bit count          = 24 bits/pixel
Compression        = 0
Size image         = 1633280 bytes
X pixels per meter = 2835
Y pixels per meter = 2835
Color used         = 0 colors
```

Pixel format [edit]

- The 1-bit per pixel (1bpp) format supports 2 distinct colors, (for example: black and white). The pixel values are stored in each bit, with the first (left-most) pixel in the most-significant bit of the first byte.[5] Each bit is an index into a table of 2 colors. An unset bit will refer to the first color table entry, and a set bit will refer to the last (second) color table entry.
- The 2-bit per pixel (2bpp) format supports 4 distinct colors and stores 4 pixels per 1 byte, the left-most pixel being in the two most significant bits (Windows CE only:[20]). Each pixel value is a 2-bit index into a table of up to 4 colors.
- The 4-bit per pixel (4bpp) format supports 16 distinct colors and stores 2 pixels per 1 byte, the left-most pixel being in the more significant nibble.[5] Each pixel value is a 4-bit index into a table of up to 16 colors.
- The 8-bit per pixel (8bpp) format supports 256 distinct colors and stores 1 pixel per 1 byte. Each byte is an index into a table of up to 256 colors.
- The 16-bit per pixel (16bpp) format supports 65536 distinct colors and stores 1 pixel per 2-byte WORD. Each WORD can define the alpha, red, green and blue samples of the pixel.
- The 24-bit per pixel (24bpp) format supports 16,777,216 distinct colors and stores 1 pixel value per 3 bytes. Each pixel value defines the red, green and blue samples of the pixel (8.8.8.0.0 in RGBAX notation). Specifically, in the order: blue, green and red (8 bits per each sample).[5]
- The 32-bit per pixel (32bpp) format supports 4,294,967,296 distinct colors and stores 1 pixel per 4-byte DWORD. Each DWORD can define the alpha, red, green and blue samples of the pixel.

File inside-the-mind-of-a-hacker-memory.bmp là loại file mỗi pixel gồm 24 bit (3 byte).

Sơ đồ

Chương trình giấu tin HiddenTreasure.exe có các đặc trưng sau:

Property	Value
File Name	C:\Users\ngahn\Downloads\HiddenTreasure\HiddenTreasure.exe
File Type	Portable Executable 32
File Info	Microsoft Visual C++ 8
File Size	10.00 KB (10240 bytes)
PE Size	10.00 KB (10240 bytes)
Created	Sunday 09 July 2017, 18.49.11
Modified	Sunday 09 July 2017, 18.49.12
Accessed	Friday 06 May 2022, 00.40.28
MD5	9F133A6B344D75EA5D0101B17983BC53
SHA-1	F0C3E286F3359E4934615F0F9B2E0415DCAAB384

Chương trình khi run nhận thêm 2 tham số:

- Tham số thứ 2: Tên của file BMP ta sẽ thực hiện giấu tin
- Tham số thứ 3: Chuỗi ký tự buffer là tin cần giấu

Chương trình chuyển buffer sang dạng xâu nhị phân, mở file bmp, giấu tin vào phần pixel array của file BMP và lưu độ dài xâu nhị phân buffer tại InfoHeader.Reserved rồi lưu file.

- Mở file bitmap và chuyển input thành dạng xâu nhị phân:

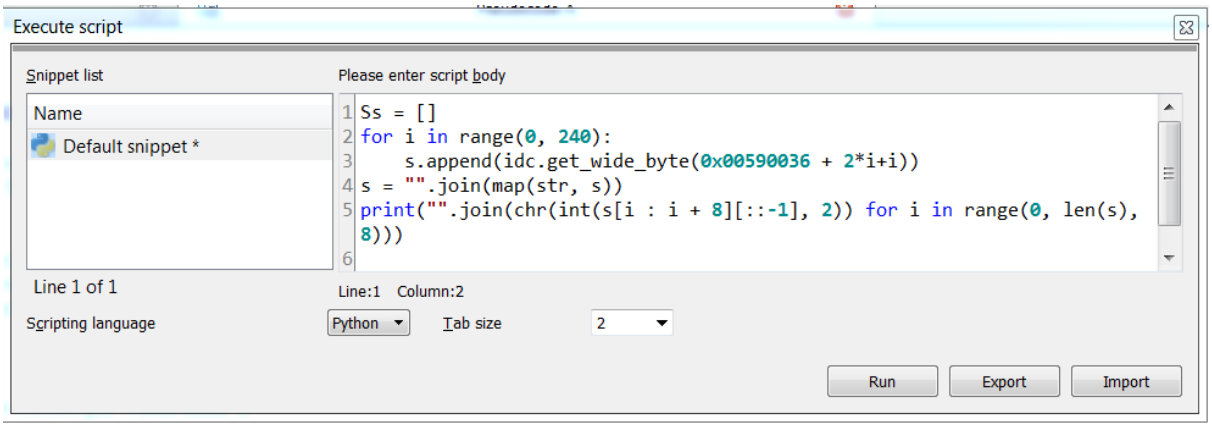
```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  if ( argc == 3 )
  {
    memset(w_inp, 0, sizeof(w_inp));
    wcstombs(w_inp, (const wchar_t *)argv[2], 0x100u);
    len = strlen(w_inp);
    if ( len )
    {
      FileW = CreateFileW((LPCWSTR)argv[1], 0xC0000000, 1u, 0, 3u, 0x80u, 0);
      v5 = FileW;
      v23 = FileW;
      if ( FileW != (HANDLE)-1 )
      {
        FileSize = GetFileSize(FileW, 0);
        if ( FileSize == -1
          || (FileMappingW = CreateFileMappingW(v5, 0, 4u, 0, 0, 0), v8 = FileMappingW, (hObject = FileMappingW) == 0) )
        {
          CloseHandle(v5);
          return 0;
        }
        lpBaseAddress = (char *)MapViewOfFile(FileMappingW, 6u, 0, 0, 0);
        if ( lpBaseAddress )
        {
          inputbin = malloc(8 * len);
          i = 0;
          j = 0;
          for ( i = 0; i < len; ++i )
          {
            chr = w_inp[i];
            inputbin[j] = chr & 1;
            inputbin[j + 1] = (chr >> 1) & 1;
            inputbin[j + 2] = (chr >> 2) & 1;
            inputbin[j + 3] = (chr >> 3) & 1;
            inputbin[j + 4] = (chr >> 4) & 1;
            inputbin[j + 5] = (chr >> 5) & 1;
            inputbin[j + 6] = (chr >> 6) & 1;
            inputbin[j + 7] = (chr >> 7) & 1;
            j += 8;
          }
```

- Lưu rải rác các bit của xâu nhị phân vào pixel array, lưu lại file bitmap:

```
43          if ( *(_WORD *)lpBaseAddress == 0x4D42 && FileSize >= *(_DWORD *)(lpBaseAddress + 2) )// BMP file signature = 0x4D42
44          {
45            FileSize = *(_DWORD *)(lpBaseAddress + 10);
46            if ( FileSize < FileSize && j < *(_DWORD *)(lpBaseAddress + 34) )
47            {
48              index = 0;
49              width = *(_DWORD *)(lpBaseAddress + 18);
50              height = *(_DWORD *)(lpBaseAddress + 22);
51              iHeader = &lpBaseAddress[FileSize];
52              h = 0;
53              *((_WORD *)lpBaseAddress + 3) = j;
54              h = 0;
55              if ( j )
56              {
57                wi = 0;
58                wi = 0;
59                do
60                {
61                  if ( h >= height )
62                    break;
63                  for ( w = 0; index < j; iHeader[2 * w1 + w1] = bin )
64                  {
65                    if ( w >= width )
66                      break;
67                    w1 = wi + w++;
68                    bin = inputbin[index++];
69                  }
70                  ++h;
71                  wi += 3 * width;
72                }
73                while ( index < j );
74              }
75            }
76          }
77          free(inputbin);
78          UnmapViewOfFile(lpBaseAddress);
79          CloseHandle(hObject);
80          CloseHandle(v23);
81        }
82        else
83        {
84          CloseHandle(v8);
85          CloseHandle(v5);
86        }
87      }
88    }
89  }
```

Debug đi vào ImageHeader để lấy được độ dài chuỗi bit đầu vào là 240 bit, execute script bên trong IDA để tách lấy dữ liệu và chuyển về dạng ascii:



Flag: flag{dont_forget_the_treasure}